UNIVERSITY INSTITUTE of COMPUTING
Asia's Fastest Growing University

NAAC GRADE A+ ACCREDITED UNIVERSITY

Worksheet 7

Student Name:Aryan Tomar **UID:25MCI10167**
Branch:MCA (AI&ML) **Section/Group:1/A**
Semester:2nd **Date of Performance:31/03/2026**
Subject Name:- DBMS LAB **Subject Code: 25CAP-652**

1. Aim of the Session

Implementation of JOINS in PostgreSQL

2. Software Requirements

- PostgreSQL (Database Server)
- pgAdmin
- Windows Operating System

3. Objective of the Session

- To understand different types of SQL JOINs (INNER, LEFT, RIGHT, CROSS)
- To retrieve and combine data from multiple related tables
- To identify matched and unmatched records using joins
- To apply JOIN operations in real-world database scenarios

4. Practical / Experiment Steps

Prerequisite Understanding

- Basic knowledge of DBMS concepts (tables, rows, columns)
- Understanding of SQL commands (SELECT, WHERE, etc.)
- Knowledge of primary key and foreign key relationships
- Basic idea of relational databases and table linking

5. Procedure of the Practical

(i) Start the system and log in to the computer.

(ii) Open PostgreSQL software.

iii) Create and select the database.

Create database Practical7;

(iv) Create table using DDL command.

```
CREATE TABLE departments (
   department_id INT PRIMARY KEY,
   department_name VARCHAR(50)
);

CREATE TABLE students (
   student_id INT PRIMARY KEY,
   name VARCHAR(50),
   department_id INT,
```

```
    FOREIGN KEY (department_id) REFERENCES departments(department_id)
);

CREATE TABLE courses (
    course_id INT PRIMARY KEY,
    course_name VARCHAR(50)
);

CREATE TABLE enrollments (
    student_id INT,
    course_id INT,
    PRIMARY KEY (student_id, course_id),
    FOREIGN KEY (student_id) REFERENCES students(student_id),
    FOREIGN KEY (course_id) REFERENCES courses(course_id)
);
```

(v) Insert records into the table.

```
INSERT INTO departments VALUES
(1, 'Computer Science'),
(2, 'Mechanical'),
(3, 'Electrical');
```

```
INSERT INTO students VALUES
(1, 'Aryan', 1),
(2, 'Rohit', 2),
(3, 'Priya', 1),
(4, 'Simran', 3),
(5, 'Karan', NULL);

INSERT INTO courses VALUES
(101, 'DBMS'),
(102, 'OS'),
(103, 'Maths'),
(104, 'Physics');

INSERT INTO enrollments VALUES
(1, 101),
(1, 102),
(2, 103),
(3, 101);
```

(vi) Display all records.

select * from students;

	student_id [PK] integer	name character varying (50)	department_id integer
1	1	Aryan	1
2	2	Rohit	2
3	3	Priya	1
4	4	Simran	3
5	5	Karan	[null]

select * from department;

	department_id [PK] integer	department_name character varying (50)
1	1	Computer Science
2	2	Mechanical
3	3	Electrical

select * from courses;

	course_id [PK] integer	course_name character varying (50)
1	101	DBMS
2	102	OS
3	103	Maths
4	104	Physics

select * from enrollments;

	student_id [PK] integer	course_id [PK] integer
1	1	101
2	1	102
3	2	103
4	3	101

Step 1: Write queries to list students with their enrolled courses (INNER JOIN).

```
select
s.student_id,s.name,c.course_name
from students s
left join enrollments e on
e.student_id=e.student_id
left join courses c on
e.course_id=c.course_id;
```

	student_id integer	name character varying (50)	course_name character varying (50)
1	1	Aryan	DBMS
2	1	Aryan	OS
3	1	Aryan	Maths
4	1	Aryan	DBMS
5	2	Rohit	DBMS
6	2	Rohit	OS
7	2	Rohit	Maths
8	2	Rohit	DBMS
9	3	Priya	DBMS
10	3	Priya	OS
11	3	Priya	Maths
12	3	Priya	DBMS
13	4	Simran	DBMS
14	4	Simran	OS
15	4	Simran	Maths
16	4	Simran	DBMS
17	5	Karan	DBMS
18	5	Karan	OS
19	5	Karan	Maths
20	5	Karan	DBMS

Step 2: Find students not enrolled in any course (LEFT JOIN).

```
select
s.student_id,s.name
from
students s
left join enrollments e on
s.student_id =e.student_id
where e.student_id is null;
```

	student_id [PK] integer	name character varying (50)
1	5	Karan
2	4	Simran

Step 3: Display all courses with or without enrolled students (RIGHT JOIN).

```
SELECT c.course_id, c.course_name, s.name
FROM courses c
LEFT JOIN enrollments e ON c.course_id = e.course_id
LEFT JOIN students s ON e.student_id = s.student_id;
```

	course_id integer	course_name character varying (50)	name character varying (50)
1	101	DBMS	Aryan
2	102	OS	Aryan
3	103	Maths	Rohit
4	101	DBMS	Priya
5	104	Physics	[null]

Step 4: Show students with department info using SELF JOIN or multiple joins.

```
SELECT s.student_id, s.name, d.department_name

FROM students s

LEFT JOIN departments d ON s.department_id = d.department_id
```

	student_id integer	name character varying (50)	department_name character varying (50)
1	1	Aryan	Computer Science
2	2	Rohit	Mechanical
3	3	Priya	Computer Science
4	4	Simran	Electrical
5	5	Karan	[null]

Step 5: Display all possible student-course combinations (CROSS JOIN). (Oracle, SAP, IBM, Microsoft)

```
SELECT s.name, c.course_name

FROM students s

CROSS JOIN courses c;
```

	name character varying (50)	course_name character varying (50)
1	Aryan	DBMS
2	Rohit	DBMS
3	Priya	DBMS
4	Simran	DBMS
5	Karan	DBMS
6	Aryan	OS
7	Rohit	OS
8	Priya	OS
9	Simran	OS
10	Karan	OS
11	Aryan	Maths
12	Rohit	Maths
13	Priya	Maths
14	Simran	Maths
15	Karan	Maths
16	Aryan	Physics
17	Rohit	Physics
18	Priya	Physics
19	Simran	Physics
20	Karan	Physics

6. I/O Analysis (Input / Output)

Input:

- Students table (student_id, name, department_id)
- Courses table (course_id, course_name)
- Enrollments table (student_id, course_id)
- Departments table (department_id, department_name)
- Relationships using primary key and foreign key

Output:

- Students with their enrolled courses
- Students not enrolled in any course
- All courses with or without students
- Students with department details
- All possible student-course combinations

7. Learning Outcomes

- Understanding different types of SQL JOINs (INNER, LEFT, RIGHT, CROSS)
- Ability to combine data from multiple related tables
- Skill to identify matched and unmatched records using joins
- Improved query writing and data retrieval skills
- Ability to apply JOIN concepts in real-world database problems